
SENTIMENT ANALYSIS SERVICE

Báo cáo phân tích kỹ thuật doanh nghiệp

Phạm vi: reverse engineering repository, review kiến trúc, system audit, và technical due diligence

Ngày đánh giá	2026-06-06
Thủ tự bằng chứng	Mã nguồn, cấu hình, hạ tầng, test, tài liệu cũ
Runtime chính	FastAPI + model runtime trong cùng process
UI chính	Angular SPA sau Nginx
Chuoi ML chính	DVC, HuggingFace, PEFT/LoRA, MLflow, ONNX Runtime

Tech Lead	Le Quang Tuyen
ML Engineer	Duong Binh An
AI Engineer	Do Quoc Trung
Solution Engineer	Duong Hong Quan
Backend Engineer	Pham Duc Long
UI/UX Engineer	Nguyen Le Hong Nhi

Phương pháp đánh giá

Báo cáo được tạo từ việc đọc mã nguồn và cấu hình trong repository. Những gì không có trong code được đánh dấu rõ là Implemented, Partially Implemented, Mocked, Deprecated, hoặc Not Implemented. Mỗi kết luận đều giới hạn theo bằng chứng tồn tại trong repo ngày 2026-06-06.

Contents

Executive Report	3
1 Section 1 - Executive Technical Summary	3
1.1 Mục đích hệ thống	3
1.2 Phạm vi kỹ thuật	3
1.3 Tổng quan công nghệ	4
1.4 Năng lực cốt lõi	4
1.5 Technical Maturity Assessment	4
1.6 System Classification	4
2 Section 2 - System Architecture Analysis	4
2.1 Context Diagram	5
2.2 Trust Boundary Diagram	5
2.3 Architecture Overview	5
2.4 Layer Assessment	5
2.5 Startup Sequence	6
2.6 Prediction Sequence	6
2.7 Batch Sequence	6
2.8 Evaluation Sequence	6
3 Section 3 - Repository Structure Analysis	6
3.1 Repository Structure Diagram	6
3.2 Major Directory Analysis	7
3.3 Critical Path and Runtime Relevance	7
3.4 Directory Assessment	7
3.5 Dependency Graph	7
3.6 Module Interaction Diagram	8
4 Section 4 - Execution Flow Analysis	8
4.1 Workflow Matrix	8
4.2 Primary Request Sequence	9
4.3 Explain Sequence	9
4.4 Training to Serving Sequence	9
5 Section 5 - Frontend Analysis	10
5.1 Architecture	10
5.2 Component and State Model	10
5.3 Components and Services	10
5.4 Frontend Data Flow	10
5.5 Key Findings	10
5.6 API Integration	10
5.7 UI Interaction Flows	11

5.8	Strengths, Weaknesses, Risks	11
6	Section 6 - Backend Analysis	11
6.1	FastAPI Architecture	11
6.2	Routing and Dependency Injection	11
6.3	API Surface	11
6.4	Contracts and Validation	12
6.5	Backend Control Flow	12
6.6	Middleware and Background Tasks	12
6.7	Background Tasks and Long-running Work	12
6.8	Metrics Collection	12
7	Section 7 - AI Inference Analysis	13
7.1	Model Loading Strategy	13
7.2	Inference Pipeline	13
7.3	Stage-by-Stage Assessment	13
7.4	Stage Analysis	13
7.5	Important Inference Findings	13
7.6	Inference Optimization Status	13
8	Section 8 - ML Pipeline Analysis	14
8.1	DVC Pipeline Graph	14
8.2	DVC Pipeline Overview	14
8.3	Stage Assessment	14
8.4	Pipeline Stage Analysis	14
9	Section 9 - Data Flow Analysis	15
9.1	End-to-End Data Lineage	15
9.2	Data Lineage	15
9.3	Artifact Lifecycle	15
9.4	Artifact Inventory	15
9.5	Transformation Logic	15
10	Section 10 - API Analysis	16
10.1	Endpoint Inventory	16
10.2	Request and Response Summary	16
10.3	Request and Failure Behavior	16
11	Section 11 - Deployment Analysis	16
11.1	Compose Topology	16
11.2	Deployment Diagram	17
11.3	Container and Runtime Notes	17
11.4	Service Inventory	17
11.5	Dockerfile Findings	17

11.6 Environment Variables and Startup Order	17
11.7 Deployment Constraints	18
12 Section 12 - Observability Analysis	18
12.1 Logging and Metrics	18
12.2 Metrics Inventory	18
12.3 Telemetry Flow	18
12.4 Implemented Coverage	18
12.5 Monitoring Coverage and Gaps	18
13 Section 13 - Security Analysis	18
13.1 Security Findings	18
14 Section 14 - Performance Analysis	19
14.1 Observed Architecture Constraints	19
14.2 Performance Risk Matrix	19
14.3 Performance Profile	19
14.4 Evidence-backed Observations	19
15 Section 15 - Failure Mode Analysis	20
15.1 Failure Tree Overview	20
15.2 Failure Tree	20
15.3 Failure Summary	20
15.4 Failure Mode Table	20
16 Section 16 - Technical Debt Analysis	21
16.1 Debt Register	21
17 Section 17 - Scalability Analysis	21
17.1 Application Scalability	21
17.2 Model Scalability	21
17.3 Operational Scalability	21
17.4 Primary Bottlenecks	21
17.5 Scalability View	21
18 Section 18 - Architectural Assessment	22
18.1 Scorecard	22
19 Section 19 - Recommendations	22
19.1 Immediate Improvements	22
19.2 Short- to Long-term Roadmap	22
20 Section 20 - Final Technical Conclusion	23
20.1 Major Strengths	23
20.2 Major Weaknesses	23

20.3 Critical Risks	23
20.4 Production Readiness and Final Verdict	23

Executive Report

Repository này thực thi một hệ thống sentiment analysis có hai mặt rõ ràng: online inference plane và offline ML lifecycle plane. Mặt online gồm Angular frontend, Nginx reverse proxy, FastAPI backend, và một model orchestration layer chạy trong cùng process để phục vụ predict, aspect extraction, sarcasm detection, SHAP explainability, health, metrics, và batch CSV đóng gói. Mặt offline gồm DVC pipeline cho download dữ liệu, tiền xử lý, validation, finetuning, evaluation, ONNX export, benchmark, và MLflow tracking.

Điểm mạnh lớn nhất là tính end-to-end có thật ở cấp repository, abstraction hợp lý thông qua `contracts/model_interface.py`, monitoring có mặt, và đường đi từ training đến serving bằng artifact ONNX. Điểm yếu lớn nhất là control plane: không có authentication, không có authorization, batch status là mock, evaluation status nằm trong memory, và các path nặng vẫn cùng chia sẻ một process.

Hệ thống phù hợp nhất với nhận diện **Hybrid Architecture: modular monolith runtime kết hợp workflow ML platform**. Đây không phải hệ thống microservice phân tán. Backend là một deployable FastAPI service sở hữu routing, validation, và orchestration model. Xung quanh nó là frontend container, monitoring services, và bộ script/pipeline ML cùng sống trong một repository.

Tóm tắt maturity

Phân loại: Hybrid Architecture / Modular Monolith có workflow ML platform

Mức trưởng thành hiện tại: Advanced MVP

Production readiness: chỉ phù hợp demo nội bộ hoặc pilot bị giới hạn chặt

Kết luận kỹ thuật tổng thể: kiến trúc đầy đủ cho học thuật và demo, nhưng chưa đủ hardening doanh nghiệp

1 Section 1 - Executive Technical Summary

1.1 Mục đích hệ thống

Hệ thống biến văn bản tự do thành kết quả sentiment có cấu trúc cho hai kiểu sử dụng: tương tác theo từng câu và xử lý CSV hàng loạt. Đồng thời repo cũng chứng minh một dây chuyền AI ứng dụng hoàn chỉnh: data acquisition, preprocessing, validation, finetuning, evaluation, export artifact, serving, và monitoring.

1.2 Phạm vi kỹ thuật

- giao diện Angular và luồng batch upload trong `app/sentiment-analysis-chatbot`
- backend API trong `src/main.py`
- inference orchestration trong `src/model/`
- data, training, evaluation, export trong `src/data/`, `src/training/`, `src/scripts/`
- Docker, Compose, Prometheus, Grafana, MLflow cho deployment và observability

1.3 Tổng quan công nghệ

Tầng	Công nghệ
Frontend	Angular 17, TypeScript, signals, RxJS, Nginx
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic
Inference	Transformers, PEFT, ONNX Runtime, SHAP, PyTorch
Data và training	Pandas, datasets, scikit-learn, DVC, MLflow
Observability	Prometheus, Grafana, custom middleware metrics
Dong goi	Docker multi-stage, Docker Compose

1.4 Năng lực cốt lõi

Năng lực	Trạng thái	Bằng chứng
Dự đoán từng câu	Implemented	POST/predict trong src/main.py
Explainability	Implemented	POST/explain gọi get_shap_explanation()
Batch CSV	Implemented	POST/batch_predict trả kết quả trực tiếp
Batch job tracking bền vững	Mocked	GET/batch_status/{job_id} trả payload có định
Kích hoạt evaluation	Partially Implemented	/evaluate chạy background thread và lưu state trong memory
Training pipeline	Implemented	run_finetuning.py, DVC stages, task configs
ONNX export	Implemented	export_onnx.py, onnx_exporter.py
Monitoring	Partially Implemented	Prometheus và Grafana có sẵn; tracing và structured logs chưa có
Authentication và authorization	Not Implemented	không có middleware hay gateway rule nào

1.5 Technical Maturity Assessment

Repo đã vượt Prototype và MVP cơ bản vì có test, contracts, monitoring, và deployment manifests. Repo chưa đạt mức production enterprise vì thiếu những control cốt lõi quanh trust boundary, durability, và workload isolation.

1.6 System Classification

Phan loại: Hybrid Architecture.

Ly do:

- backend là modular monolith: một FastAPI process sở hữu routing, validation, model orchestration, và admin-like evaluation trigger
- frontend được container hóa riêng và reverse-proxy qua Nginx
- monitoring dùng service bên ngoài, nhưng runtime ứng dụng không tách thành business services độc lập
- cùng một repo chứa cả workflow ML platform với DVC, MLflow, training, evaluation, và export

2 Section 2 - System Architecture Analysis

2.1 Context Diagram

Luồng tổng thể gồm browser, frontend Angular, reverse proxy Nginx, FastAPI runtime, model inference layer, và khối monitoring / metrics. Cấu trúc này thể hiện rõ ranh giới giữa client, application runtime, và observability plane.

2.2 Trust Boundary Diagram

Trust boundary chính nằm giữa browser và backend API, giữa backend runtime và artifact/model storage, và giữa runtime ứng dụng và dịch vụ monitoring bên ngoài. Các ranh giới này quan trọng vì current deployment vẫn expose nhiều endpoint public.

2.3 Architecture Overview

```

1 flowchart LR
2   Browser --> FE[Angular Frontend]
3   FE --> NX[Nginx Reverse Proxy]
4   NX --> API[FastAPI Runtime]
5   API --> LD[Language Detector]
6   API --> INF[BaselineModelInference]
7   INF --> ONNX[ONNX Runtime Session]
8   INF --> HF[HF / PEFT Model Path]
9   INF --> ABSA[Zero-shot ABSA Pipeline]
10  INF --> SHAP[SHAP Explainer]
11  API --> METRICS[/metrics]
12  METRICS --> PROM[Prometheus]
13  PROM --> GRAF[Grafana]
14  API --> EVAL[/evaluate]
15  EVAL --> MLFLOW[MLflow]
```

2.4 Layer Assessment

Layer	Mục đích	Dependencies	Input	Output	Failure modes	Rủi ro scale
Frontend	tuong tac nguoi dung, Angular, local state, explain va batch UX	Angular, localStorage, speech API	HttpClient, text, CSV, health browser checks	HTTP requests, rendered results	backend down, state contract cu	nam tren browser va pay-load batch lon
Backend	routing, validation, life-cycle, status surfaces	FastAPI, global model instance	Pydantic, HTTP requests, file upload	JSON, text, metrics	plain-startup load fail, errors, CSV loi, route public	mot process so huu model va nang
Inference	biến text thành labels, aspects, sarcasm, SHAP	la-Transformers, Runtime, PEFT, SHAP	ONNX text va language tokenizers,	du doan va giai thích	artifact thieu, ABSA va unsupported language, ABSA khong co cache fail	ton CPU va va
Training	tao adapters va ket qua evaluation	datasets, Trainer, MLflow, pandas	PEFT, raw CSVs va params	adapters, metrics, logs	dataset drift, imbalance, export chain loi	chi co local hoac single-container execution
Data	download, normalize, validate, prepare sets	pandas, XML parser, datasets, DVC	XML files va datasets	HF processed CSVs, reports, eval sets	source files thieu, schema loi	pipeline based governance thu cong
Monitoring	thu thap counters va tency histograms	la-prometheus_client, Prometheus, Grafana	metrics endpoint	dashboards va alerts	va scrape sai, dashboard drift	config chi nhìn thay cap API, khong co tracing

Layer	Mục đích	Dependencies	Input	Output	Failure modes	Rủi ro scale
Deployment	build và kết nối containers	Docker, Compose, Nx, DVC pull	Ng-code, credentials, artifacts	runnable stack	build leak, mismatch	secret artifact không có rollout strategy
Security	bảo vệ API, secrets, artifacts, containers	env vars và default work	net-credentials và public endpoints	enforcement policy	auth default CORS	absent, trust secret, yeu boundary

2.5 Startup Sequence

Startup flow bắt đầu từ container entrypoint, đi qua FastAPI lifespan hook, rồi tạo `BaselineModelInference` và nạp model artifacts. Nếu artifact, tokenizer, hoặc backend mode không khớp, service không đi vào trạng thái ready.

2.6 Prediction Sequence

Prediction sequence là luồng đồng bộ chính: request đi từ frontend qua backend, qua language detection, model inference, rồi trả `PredictResponse`. Đây là path có tần suất cao nhất và quyết định độ trễ cảm nhận của người dùng.

2.7 Batch Sequence

Batch sequence đọc CSV, xử lý từng dòng theo chunk, rồi trả kết quả đồng bộ thay vì job queue thật. Điều này đơn giản hóa triển khai nhưng giới hạn throughput và làm giảm khả năng retry / recovery.

2.8 Evaluation Sequence

Evaluation sequence chạy như background task trong cùng process, ghi trạng thái vào memory và đẩy metric / report ra file hoặc MLflow. Cách này đủ cho pilot, nhưng chưa phù hợp nếu cần durability hay tracing đầy đủ.

3 Section 3 - Repository Structure Analysis

3.1 Repository Structure Diagram

```

1 flowchart TD
2   ROOT[Repository Root]
3   ROOT --> APP[app/]
4   ROOT --> CONTRACTS[contracts/]
5   ROOT --> DATA[data/]
6   ROOT --> DOCS[docs/]
7   ROOT --> INFRA[infra/]
8   ROOT --> MODELS[models/]
9   ROOT --> SRC[src/]
10  ROOT --> TESTS[tests/]
11  ROOT --> DVC[dvc.yaml and params.yaml]
12  ROOT --> DOCKER[Dockerfile and docker-compose.yml]
```

3.2 Major Directory Analysis

3.3 Critical Path and Runtime Relevance

Các thư mục runtime-critical tập trung ở `src/model/`, `src/main.py`, `contracts/`, và `models/`. Các thư mục training-critical nằm ở `src/data/`, `src/training/`, và `src/scripts/`. Đây là các path quyết định serving, export, evaluation, và reproducibility.

3.4 Directory Assessment

Nhóm thư mục còn lại được phân tách theo mức độ ảnh hưởng: `app/` ảnh hưởng trực tiếp đến UX, `infra/` ảnh hưởng đến monitoring, còn `docs/` chủ yếu phục vụ audit và handoff. Cách phân chia này phù hợp với mô hình modular monolith có ML workflow đi kèm.

Thu mục	Mục đích	Criticality	Runtime vance	rele- Dependencies	Modification risk	Complexity
<code>app/</code>	Angular UI và frontend Docker build	High	Direct	backend contracts và Nginx proxy	High	Medium
<code>contracts/</code>	shared schemas, model interface, errors, examples	High	Direct	FastAPI và tests	High	Low
<code>data/</code>	raw, processed, eval, re-port artifacts	High	Indirect runtime; direct training	DVC và scripts	High	Medium
<code>docs/</code>	reports và legacy docs	Low	Indirect	none at runtime	Low	Low
<code>infra/</code>	Prometheus và Grafana provisioning	Medium	Direct	metrics endpoint và Compose DNS	Medium	Low
<code>models/</code>	adapters và ONNX artifacts	Critical	Direct	training, export, runtime load	Critical	Medium
<code>src/data/</code>	download, preprocess, validate pipeline	High	Training-critical	params và raw data	High	Medium
<code>src/model/</code>	inference, export, evaluation, device logic	Critical	Runtime-critical	model artifacts và ML libs	Critical	High
<code>src/monitoring/</code>	metric objects và middleware	Medium	Runtime-critical	FastAPI middle-ware chain	Medium	Low
<code>src/scripts/</code>	CLIs cho training, eval, export, benchmark	High	Lifecycle-critical	training và model modules	High	Medium
<code>src/training/</code>	reusable training nals và metrics	High	Lifecycle-critical	Trainer, pandas, sklearn	High	Medium
<code>tests/</code>	unit và integration evi-dence	Medium	Indirect	pytest và mocks	Medium	Medium

3.5 Dependency Graph

```

1 flowchart LR
2   APP --> CONTRACTS
3   APP --> MAIN[src/main.py]
4   MAIN --> CONTRACTS
5   MAIN --> MODEL[src/model]
6   MAIN --> MON[src/monitoring]
7   SCRIPTS[src/scripts] --> TRAIN[src/training]
8   SCRIPTS --> DATA[src/data]
9   SCRIPTS --> MODEL
10  MODEL --> MODELS
11  INFRA --> MAIN

```

3.6 Module Interaction Diagram

```

1 flowchart LR
2   ChatUI --> SentimentService
3   SentimentService --> PredictAPI
4   SentimentService --> ExplainAPI
5   SentimentService --> BatchAPI
6   PredictAPI --> BaselineModelInference
7   ExplainAPI --> BaselineModelInference
8   BatchAPI --> BaselineModelInference
9   BaselineModelInference --> OnnxSession
10  BaselineModelInference --> ABSAPipeline
11  BaselineModelInference --> ShapExplainer

```

4 Section 4 - Execution Flow Analysis

4.1 Workflow Matrix

Workflow	Entry point	Execution path	Key files	Outputs	Status
Startup	FastAPI lifespan	tao khoi tao BaselineModelInference, preload ABSA	ModelConfig, src/main.py, tao model/baseline.py	loaded model state hoac startup fail	Implemented
Health check	GET/health	dependency injection tra payload	roi src/main.py, contracts/schemas.py	health JSON	Implemented
Prediction	POST/ predict	validate, detect language, predict single, record tency	them language_ detector.py	PredictResponse	Implemented
Explanation	POST/ explain	validate, detect language, build SHAP on demand	baseline.py	ExplainResponse	Implemented
Batch processing	POST/batch_ predict	doc CSV, cap 500 dong, thread off sync batch inference	src/main.py	danh sach ket qua	Implemented
Batch status	GET/batch_ status/ \{id\}	tra du lieu co dinh	src/main.py	payload hoan thanh gia	Mocked
Evaluation	POST/ evaluate va GET/ evaluate/ status	background task logic evaluate, model/evaluate.py luu state trong memory	goi src/main.py, src/evaluate.py	MLflow logs va status fields	Partially Implemented
Training	python-msrc.load scripts. run_ finetuning	raw data, dedup, split, oversample, train LoRA adapter	src/scripts/run_ finetuning.py, src/training/	adapter dirs va MLflow run	Implemented
Finetuned evaluation	python-msrc.load scripts. evaluate_ finetuned	finetuned batch eval, ghi reports	config, evaluate_ finetuned.py, training/metrics.py	metrics JSON	Implemented
ONNX export	python-msrc.merge scripts. export_onnx	PEFT adapter va export FP32 + INT8	export_onnx.py, onnx_exporter.py	ONNX dirs	Implemented

Workflow	Entry point	Execution path	Key files	Outputs	Status
Deployment	dockercompose	build backend, frontend, Dockerfiles va Compose monitoring, MLflow, optional trainer		multi-container stack	Implemented

4.2 Primary Request Sequence

```

1 sequenceDiagram
2   participant U as User
3   participant FE as Angular
4   participant NX as Nginx
5   participant API as FastAPI
6   participant LD as LanguageDetector
7   participant INF as BaselineModelInference
8   U->>FE: gui text
9   FE->>NX: POST /api/predict
10  NX->>API: forward request
11  API->>LD: detect neu thieu lang
12  API->>INF: predict_single(text, lang)
13  INF-->>API: sentiment, confidence, aspects, sarcasm
14  API-->>NX: PredictResponse
15  NX-->>FE: JSON
16  FE-->>U: hien thi ket qua

```

4.3 Explain Sequence

```

1 sequenceDiagram
2   participant FE as Angular
3   participant API as FastAPI
4   participant INF as BaselineModelInference
5   participant SHAP as SHAP Explainer
6   FE->>API: POST /api/explain
7   API->>INF: get_shap_explanation(text, lang)
8   INF->>SHAP: tao explainer va tinh values
9   SHAP-->>INF: tokens, values, base value
10  INF-->>API: SHAPResult
11  API-->>FE: ExplainResponse

```

4.4 Training to Serving Sequence

```

1 sequenceDiagram
2   participant DVC
3   participant DATA as Data Pipeline
4   participant TRAIN as Finetuning
5   participant EVAL as Evaluation
6   participant EXP as ONNX Export
7   participant RT as Runtime Image
8   DVC->>DATA: download, preprocess, validate
9   DATA->>TRAIN: normalized CSV inputs
10  TRAIN->>EVAL: adapters and metrics

```

```

11  TRAIN->>EXP: trained adapter directories
12  EXP->>RT: ONNX artifacts
13  RT->>RT: startup load

```

5 Section 5 - Frontend Analysis

5.1 Architecture

Frontend là một Angular application dùng standalone components. State được lưu chủ yếu bằng Angular signals trong `SentimentAnalysisService`. Không có external state manager như NgRx.

5.2 Component and State Model

Component tree xoay quanh app shell, chat input, message bubble, và batch upload modal. State chủ yếu tồn tại trong service singleton và browser storage, nên coupling giữa UI state và API contract khá chặt.

5.3 Components and Services

Artifact	Vai tro	Nhan xet
<code>app.component.ts</code>	shell chinh, health polling, render chat history, modal toggles	component lon, giao UI, chua nhieu orchestration
<code>sentiment-analysis.service.ts</code>	API client va message state, explain trigger logic	service trung tam, coupling cao voi payload
<code>chat-input.component.ts</code>	text entry, speech-to-text, language picker, batch button	nhieu tinh nang; language support vuot backend support
<code>message-bubble.component.ts</code>	render kết quả sentiment và SHAP bars	explainability UI được làm trực tiếp
<code>batch-upload.component.ts</code>	upload CSV, tóm tắt kết quả, export CSV	batch UX đồng bộ, không dùng job polling

5.4 Frontend Data Flow

```

1  flowchart LR
2    Input[Chat Input]
3    Service[SentimentAnalysisService]
4    API[REST endpoints]
5    State[Signal message state]
6    View[Message bubbles and batch modal]
7    Input --> Service --> API --> State --> View

```

5.5 Key Findings

5.6 API Integration

Frontend gọi trực tiếp các endpoint public qua HTTP client, chủ yếu là `/health`, `/predict`, `/explain`, và `/batch_predict`. Không có gateway-level abstraction riêng, nên contract drift sẽ được cảm nhận ngay ở UI.

5.7 UI Interaction Flows

UI flow gồm chat text, speech-to-text, batch upload, và xem explainability. Các flow này đều chạy trên cùng state service, vì vậy batch và chat share nhiều logic hiển thị và error handling.

- health polling có that và điều khiển loading overlay
- batch processing UI được implement và kỳ vọng kết quả sync
- explainability UI được implement và vẽ SHAP token contributions
- component phát ra lang, nhưng `handleSendText()` và `predict()` gán như bỏ qua giá trị này cho predict thông thường
- language picker hiện nhiều ngôn ngữ, trong khi backend chỉ support en và vi

5.8 Strengths, Weaknesses, Risks

Loại	Danh giá
Strengths	trải nghiệm giao diện chau chuốt, explain UI có that, local persistence, startup có health awareness
Weaknesses	top-level component lộn, service gom cả transport và display logic, frontend tests mong, mismatch về language contract
Risks	UX đa ngôn ngữ dễ gây hiểu nhầm, coupling cao với bot message formatting, state chỉ nằm trên browser
Maintainability concerns	inline template lộn, UI logic và transport logic trộn lẫn, mapping payload nằm trong code tạo chuỗi hiển thị

6 Section 6 - Backend Analysis

6.1 FastAPI Architecture

Backend là một FastAPI application đơn lẻ trong `src/main.py`. Nó sử dụng:

6.2 Routing and Dependency Injection

Routing tập trung trong một file chính, với dependency injection được dùng để lấy model instance, validate request context, và gắn các route handlers vào cùng lifecycle. Điều này làm hệ thống dễ hiểu nhưng tạo một điểm tập trung lớn.

- lifespan hook để khởi tạo model
- dependency injection cho model instance
- Pydantic schemas cho request và response
- middleware để ghi Prometheus metrics
- background tasks chỉ cho evaluation

6.3 API Surface

Method	Path	Mục đích	Validation	Failure modes	Performance notes
GET	/health	readiness và supported languages	model dependency phải resolve	model not loaded sẽ re 503 tu dependency	
POST	/predict	du đoán tung câu	text 1..2000 qua schema	unsupported language, model error	lan- ABSA và sarcasm tang latency
POST	/explain	SHAP explanation	cung schema voi predict	SHAP hoặc tokenizer fail	tok- latency cao nhất
POST	/batch_predict	cham diem CSV dong bo	CSV hop le, co cot text, gioi han 500 dong	bad CSV, dong rong, inference fail	thread-offloaded nhưng van CPU-bound
GET	/batch_status/{job_id}	contract cho batch job status	chi co path string	khong co; tra gia tri co dinh	mocked
POST	/evaluate	trigger evaluation tu run-time	chan concurrent runs bang in-memory flag	thieu processed data, MLflow fail	heavy admin-like task trong public app
GET	/evaluate/status	expose in-memory evaluation state	none	reset khi restart process	re nhưng không ben
GET	/metrics	Prometheus scrape point	end- none	metrics library fail	re

6.4 Contracts and Validation

Validation mạnh ở JSON edge nhờ Pydantic schemas. Validation file yêu hơn: batch upload chỉ kiểm CSV đọc được và có cột `text`; không ép MIME type, file size ngay trong FastAPI, hay content sanitization ngoài việc trim text.

6.5 Backend Control Flow

Control flow của backend đi từ request parsing, validation, dependency resolution, inference call, đến response formatting. Những path nặng như explain và batch vẫn nằm chung event loop / process, nên error isolation còn thấp.

6.6 Middleware and Background Tasks

Chỉ có một middleware là `monitor_middleware`. Background task chỉ dùng cho evaluation, không dùng cho batch scoring hay explainability.

6.7 Background Tasks and Long-running Work

Evaluation là long-running task duy nhất được đẩy sang background thread, nhưng batch inference và SHAP vẫn chạy đồng bộ. Đây là giới hạn rõ ràng về throughput và khả năng phục hồi khi workload tăng.

6.8 Metrics Collection

Metrics được thu thập qua middleware và `/metrics`, với latency histogram và error counters là hai nguồn tín hiệu chính. Chưa có tracing hoặc request correlation nên việc điều tra theo transaction còn bị giới hạn.

7 Section 7 - AI Inference Analysis

7.1 Model Loading Strategy

Model loading strategy chọn giữa baseline, finetuned, ONNX, và INT8 ONNX dựa trên config và artifact tồn tại. Runtime ưu tiên đường ONNX khi artifacts đầy đủ, giúp giảm chi phí forward pass nhưng tăng độ phụ thuộc vào export correctness.

7.2 Inference Pipeline

```
1 flowchart LR
2   Text --> Lang[Language Resolution]
3   Lang --> Sent[Sentiment Model]
4   Sent --> Sarc[Sarcasm Model or Fallback False]
5   Sent --> ABSA[Zero-shot ABSA]
6   Sent --> Resp[PredictionResult]
7   Text --> SHAP[SHAP Explainer on Demand]
```

7.3 Stage-by-Stage Assessment

7.4 Stage Analysis

Các stage chính gồm tokenization, language detection, sentiment inference, sarcasm detection, ABSA, và SHAP. Chỉ sentiment inference là path cốt lõi có thể tối ưu tốt; ABSA và SHAP là phần gây chi phí lớn nhất và dễ trở thành bottleneck.

Stage	Input	Transformation	Output	Cost impact	Failure modes
Model loading	config mode va paths	chọn ONNX, ONNX live INT8, finetuned, hoặc baseline path	model/session state	startup nặng	artifact thiếu, export không tương thích
Tokenizer	text list	tokenization, trunca- tion, padding	tensors hoặc NumPy arrays	trung bình	tokenizer thiếu hoặc mis-match
Language detection	optional lang va text	heuristic nhận diện dấu tiếng Viet	LanguageDetectionResult	thấp	detect sai, đặc biệt với text ngắn
Sentiment inference	tokenized text	PyTorch forward pass hoặc ONNX session run	class probabili- chi phi chinh ties	chi phí chính	provider mismatch, model error
Sarcasm detection	text	them mot lan predict hoặc default false	boolean flag	tăng chi phí tung mau	sarcasm model thiếu thì silently degrade về false trong ONNX mode
ABSA	text	zero-shot detect aspect va sentiment-per- aspect	aspect list	rat cao	pipeline fail thì tra as- pects rong
SHAP	text	tạo explainer và pertur- bation sampling	token contribu- cations	cao nhất	không có tokenizer, la- tency, memory pressure

7.5 Important Inference Findings

7.6 Inference Optimization Status

Optimization hiện dựa chủ yếu vào ONNX export, batching đơn giản, và tắt bớt feature nặng trong batch path. Chưa thấy cache theo request, worker isolation, hay precomputed explainability artifacts.

- ModelConfig support baseline, finetuned, onnx, va onnx_int8
- runtime mac định la onnx thông qua fallback của MODEL_MODE
- ABSA không phải một model aspect được train riêng; nó là zero-shot classification trên tập labels có cấu hình
- language detection cho tiếng Việt chỉ là heuristic
- trong batch mode, ABSA bị bỏ qua để giữ throughput

8 Section 8 - ML Pipeline Analysis

8.1 DVC Pipeline Graph

Pipeline DVC liên kết các stage download, preprocess, validate, finetune, evaluate, export, và benchmark thành một lineage có thể lặp lại. Đây là phần mạnh nhất của workflow vì nó cung cấp reproducibility rõ ràng.

8.2 DVC Pipeline Overview

```

1 flowchart LR
2   download --> preprocess --> validate --> evaluate_baseline
3   download_sarcasm --> finetune_sarcasm --> export_onnx_sarcasm
4   download_sentiment --> prepare_eval --> finetune_sentiment -->
   evaluate_finnetuned --> export_onnx_sentiment --> benchmark_onnx

```

8.3 Stage Assessment

8.4 Pipeline Stage Analysis

Mỗi stage đều tạo artifact riêng và có failure surface riêng: download phụ thuộc nguồn ngoài, preprocess phụ thuộc schema, finetune phụ thuộc dữ liệu và params, export phụ thuộc adapter merge, benchmark phụ thuộc runtime resource. Vì vậy lỗi ở giai đoạn đầu có thể propagate sang serving.

Stage	Mục đích	Inputs	Outputs	Artifacts	Failure modes
download	parse SemEval thành raw CSVs	XML XML local trong data/external/ semeval2014	raw sentence va aspect CSVs	data/raw/*.csv	XML thiếu hoặc schema drift
preprocess	normalize va dataset	split raw sentence va aspect CSVs	processed CSVs	data/processed/	bad columns, transform fail
validate	quality gate data	processed processed CSVs va params	JSON quality report	data/reports/quality_report.json	threshold fail
download_sarcasm	lay tweet-eval irony data	HuggingFace dataset	sarcasm raw CSV	data/raw/sarcasm.csv	network hoặc dataset change
download_sentiment	lay EN va VI sentiment data	HuggingFace parquet sources	hai raw CSV	data/raw/sentiment_*.csv	dataset source fail
prepare_eval	tao held-out eval CSVs	raw sentiment CSVs	mixed-lang va VI probe sets	data/eval/	raw data missing

Stage	Mục đích	Inputs	Outputs	Artifacts	Failure modes
finetune	train LoRA adapters	raw training CSVs va params	adapter dirs va MLflow runs	models/ adapters/	imbalance, training fail
evaluate_finetuned	sinh fairness-aware re-ports	adapters va raw/eval data	JSON metrics	reports/*.json	metric mismatch hoặc missing adapter
export_onnx	đoi adapter thành artifact	finetuned adapter path	FP32 va INT8 ONNX dirs	models/onnx/	merge/export fail
benchmark_onnx	so sanh cac mode	generated ONNX va finetuned model	benchmark JSON	reports/ onnx_ benchmark. json	resource pressure

9 Section 9 - Data Flow Analysis

9.1 End-to-End Data Lineage

Data lineage bắt đầu từ XML / HF sources, đi qua normalization, validation, training split, finetuning, evaluation, rồi xuất thành adapters hoặc ONNX artifacts. Dòng chảy này cho thấy rõ đâu là raw input, đâu là runtime artifact.

9.2 Data Lineage

```

1 flowchart LR
2   Raw[Raw Data]
3   Prep[Processing]
4   Val[Validation]
5   Train[Training]
6   Eval[Evaluation]
7   Export[ONNX Export]
8   Load[Runtime Load]
9   Predict[Prediction]
10  Raw --> Prep --> Val --> Train --> Eval --> Export --> Load --> Predict

```

9.3 Artifact Lifecycle

9.4 Artifact Inventory

Artifact inventory chủ đạo gồm raw CSV, processed CSV, eval CSV, adapters, ONNX models, benchmark reports, và MLflow runs. Trong đó adapters và ONNX models là hai artifact có ảnh hưởng trực tiếp nhất đến production runtime.

9.5 Transformation Logic

Transformation logic chủ yếu là parse XML, normalize text, split dataset, map labels, train adapters, rồi export sang ONNX. Các transform này deterministic ở mức pipeline, nhưng vẫn nhạy với schema drift và dataset quality.

Artifact type	Vị trí	Vòng đời
SemEval raw CSVs	data/raw/sentences.csv, data/raw/aspects.csv	sinh từ XML, được preprocessing sử dụng
Processed training data	data/processed/	sinh bởi pipeline, được baseline evaluation và runtime-side evaluation sử dụng
Sentiment và sarcasm raw CSVs	data/raw/sentiment_*.csv, data/raw/sarcasm.csv	download để finetune và evaluate
Eval CSVs	data/eval/	sinh bởi prepare_eval.py, được finetuned evaluation sử dụng
Adapters	models/adapters/	sinh bởi finetuning, được evaluate và export ONNX sử dụng
ONNX models	models/onnx/	sinh bởi export, được runtime inference sử dụng
MLflow runs	mlruns/ hoặc tracking URI bên ngoài	sinh trong training và evaluation, được engineers xem
JSON reports	data/reports/ và reports/	sinh trong validate, eval, benchmark

10 Section 10 - API Analysis

10.1 Endpoint Inventory

Endpoint inventory của backend tập trung vào health, predict, explain, batch predict, batch status, evaluate, evaluate status, và metrics. Tất cả endpoint này hiện cùng nằm trong một public surface.

10.2 Request and Response Summary

10.3 Request and Failure Behavior

Các endpoint đồng bộ trả lỗi nhanh khi validate fail, nhưng các path nặng có thể kéo dài latency hoặc làm nghẽn process. Failure behavior của batch status và evaluate status khác nhau rõ: một cái mocked, một cái in-memory và không bền.

Endpoint	Auth status	Schema source	Ghi chú
GET/health	public	HealthResponse	bao supported languages từ model config
POST/predict	public	PredictRequest/PredictResponse	business endpoint quan trọng nhất
POST/explain	public	ExplainRequest/ExplainResponse	đặt, không có throttling
POST/batch_predict	public	multipart + BatchPredictResponse	đồng bộ du docs ở nơi khác để như async job
GET/batch_status/{job_id}	public	BatchStatusResponse	mocked endpoint
POST/evaluate	public	ad hoc JSON	admin-like action bị expose
GET/evaluate/status	public	ad hoc JSON	state chỉ nằm trong memory
GET/metrics	public	Prometheus text	scrape endpoint không auth

11 Section 11 - Deployment Analysis

11.1 Compose Topology

Compose topology ghép frontend, backend, Prometheus, Grafana, MLflow, và trainer profile vào một stack local-first. Cách deploy này đơn giản nhưng không có progressive delivery hay rescheduling

semantics.

11.2 Deployment Diagram

```
1 flowchart TD
2   subgraph Compose
3     FE[frontend :80]
4     API[fastapi_app :8000]
5     PROM[prometheus :9091]
6     GRAF[grafana :3000]
7     MLF[mlflow :5005]
8     TR[trainer profile]
9   end
10  FE --> API
11  PROM --> API
12  GRAF --> PROM
13  API --> MLF
14  TR --> MLF
```

11.3 Container and Runtime Notes

11.4 Service Inventory

Service inventory gồm frontend container, backend API container, monitoring stack, MLflow service, và optional training container. Mỗi service có một vai trò rõ nhưng mức độ isolation còn hạn chế vì tất cả nằm trong cùng compose boundary.

11.5 Dockerfile Findings

Dockerfiles cho thấy pattern multi-stage khá chuẩn, nhưng runtime image vẫn phụ thuộc vào artifact có sẵn và một số build-time credential path. Điều này đủ cho local stack nhưng cần hardening nếu dùng cho deployment doanh nghiệp.

11.6 Environment Variables and Startup Order

Startup order phụ thuộc nhiều vào env vars như model mode, artifact paths, monitoring config, và DVC credentials. Nếu biến môi trường hoặc artifact chưa sẵn sàng, service sẽ fail sớm thay vì degrade gracefully.

- backend Dockerfile là multi-stage: build dependencies, DVC model pull, final runtime
- runtime image pre-download HuggingFace assets rồi bật offline mode
- runtime image dùng DVC build args cho DagsHub credentials; chạy được nhưng rủi ro
- frontend container proxy /api/ về backend và mở rộng upload/time limits
- Compose có Prometheus, Grafana, MLflow, và một training profile tùy chọn

11.7 Deployment Constraints

- Compose là đường orchestration duy nhất được commit
- không có manifests Kubernetes hay managed deployment
- không có rolling release, canary, hay blue-green workflow
- startup phụ thuộc model artifacts và downloaded base assets phải tồn tại và tương thích

12 Section 12 - Observability Analysis

12.1 Logging and Metrics

Observability hiện có metrics rõ hơn logging. Metrics endpoint và Prometheus scrape giúp nhìn thấy health và latency, nhưng log format chưa đủ chuẩn để truy lỗi theo request.

12.2 Metrics Inventory

Metrics inventory gồm request counters, latency histograms, và model inference timers. Đây là nền tảng tốt, nhưng chưa đủ để cover business-level SLI hay error budget.

12.3 Telemetry Flow

Telemetry flow đi từ middleware đến Prometheus scrape, rồi sang Grafana dashboard và alert rules. Flow này thiếu tracing nên chỉ cho view theo service, không theo transaction.

12.4 Implemented Coverage

12.5 Monitoring Coverage and Gaps

Monitoring hiện quan sát được API health, request rate, latency, và một phần error rate. Gaps lớn là alert routing, tracing, SLO/SLI definitions, và coverage cho batch / explain workflows.

Area	Status	Bằng chứng
Request metrics	Implemented	counters và histograms trong <code>src/monitoring/metrics.py</code>
Model inference latency metric	Implemented	<code>MODEL_INFERENCE_LATENCY</code> theo language
Prometheus scrape config	Implemented	<code>infra/prometheus/prometheus.yml</code>
Alert rules	Partially Implemented	chỉ có 2 alerts: high 5xx và high latency
Grafana provisioning	Implemented	datasource và dashboard provisioning files tồn tại
Structured logging	Partially Implemented	Python logging có, nhưng không có request log format rõ ràng
Tracing và correlation IDs	Not Implemented	không có OpenTelemetry hay request-id propagation
Alert routing và paging	Not Implemented	không có Alertmanager config hay notification target

13 Section 13 - Security Analysis

13.1 Security Findings

Finding	Severity	Bằng chứng	Tác động	Khuyến nghị
Không có authentication	Critical	không có auth dependency hay gateway auth	public có thể lạm dụng predict, explain, evaluate, metrics	thêm gateway và app authentication
Không có authorization	Critical	public và admin-like routes chung	heavy hoặc privileged actions bị expose	tách /evaluate và operator functions
Permissive CORS	High	allow_origins=["*"]	mọi browser origin đều gọi được API	giới hạn allowlist theo môi trường
Grafana password mặc định	High	GF_SECURITY_ADMIN_PASSWORD=admin	dashboard bị truy cập trái phép	đưa vào secret và rotate
Build-time secrets cho DVC pull	High	Docker build args cho DagsHub credentials	credentials để lộ qua build system hoặc cache	dùng runtime secret injection hoặc artifact registry
Không có rate limiting	High	không có throttle logic	denial-of-wallet explain hoặc batch	thêm quota và rate controls
File upload controls yếu	Medium	chỉ check CSV parse và content	upload quá lớn hoặc sai định dạng gây stress backend	size, MIME, và row policy
Artifact integrity ngầm định	Medium	không có checksum hay signature validation	có thể serve nhầm artifact	thêm checksum và promotion gate
Container hardening hạn chế	Medium	images mặc định, không thấy non-root runtime config	attack surface rộng hơn	chạy non-root và scan image

14 Section 14 - Performance Analysis

14.1 Observed Architecture Constraints

Constraints chính đến từ in-process model owner, synchronous explainability, và batch path không có worker queue. Điều này làm performance phụ thuộc trực tiếp vào CPU và memory của một container duy nhất.

14.2 Performance Risk Matrix

Risk matrix nghiêng mạnh về explain và ABSA: khi request mix chuyển từ predict sang explain, latency và memory pressure tăng nhanh hơn tuyến tính. ONNX giảm cost dự đoán nhưng không loại bỏ chi phí SHAP.

14.3 Performance Profile

- startup cost bị chi phí bởi model/session loading và ABSA preload
- cost đo đơn vị câu hỏi ở mức vừa trong ONNX mode, nhưng tăng mạnh nếu có ABSA và sarcasm
- explain cost là cao nhất vì SHAP được tạo on demand
- batch throughput được cải thiện nhờ chunked inference và bỏ qua ABSA, nhưng vẫn single-host và in-process

14.4 Evidence-backed Observations

- benchmark tests assert mức tối thiểu 100 samples/sec cho sentiment-only batch throughput

- Prometheus warning threshold xem average inference tren 500 ms la degrade
- Docker Compose tang CPU va memory limits cho backend de phục vụ inference
- exact startup và memory telemetry chưa được commit thành runtime measurements

15 Section 15 - Failure Mode Analysis

15.1 Failure Tree Overview

Fault domains chia theo startup, prediction, explain, batch, training, deployment, và monitoring. Mỗi domain có root causes riêng nhưng đều có chung một đặc điểm: artifact path, config, và runtime process là ba điểm dễ hỏng nhất.

15.2 Failure Tree

```

1 flowchart TD
2   A[Startup failure] --> A1[missing model artifacts]
3   A --> A2[tokenizer or HF asset mismatch]
4   B[Prediction failure] --> B1[unsupported language]
5   B --> B2[model exception]
6   C[Explain failure] --> C1[SHAP cost or memory pressure]
7   C --> C2[tokenizer unavailable]
8   D[Batch failure] --> D1[invalid CSV]
9   D --> D2[long-running CPU-bound inference]
10  E[Training failure] --> E1[missing raw data]
11  E --> E2[dataset imbalance or bad labels]
12  F[Deployment failure] --> F1[bad credentials or DVC pull]
13  F --> F2[Compose config drift]

```

15.3 Failure Summary

15.4 Failure Mode Table

Failure mode table cho thấy startup và deployment phụ thuộc artifact / credential correctness, còn prediction và explain phụ thuộc mode, tokenizer, và compute headroom. Batch và evaluation lại chịu rủi ro lớn từ synchronous CPU work và state không bền.

Area	Triệu chứng	Recovery path
Startup	health 503 hoặc container crash	xác minh artifact paths, mode, tokenizer assets, image build inputs
Prediction	400/500 responses hoặc aspects rong	xác minh language handling và model load, xem warning logs
Explanation	response rat chậm hoặc fail	tách explain path, throttle usage, kiểm tra tokenizer và SHAP deps
Batch processing	request timeout hoặc parser error	xác minh CSV input, giảm payload size, cân worker architecture
Evaluation	status stuck hoặc reset sau restart	nhìn ra state nằm trong memory, rerun evaluation, chuyển sang job tracking bên vùng
Training	stage fail hoặc artifact rong	rerun DVC stages, kiểm tra raw files, xác minh params và task configs
Deployment	build hoặc stack launch fail	xem Compose logs, secret inputs, DVC pull behavior, network aliases
Monitoring	dashboard rỗng hoặc không có alert	test /metrics, Prometheus target status, Grafana datasource provisioning

16 Section 16 - Technical Debt Analysis

16.1 Debt Register

Debt register tập trung vào security, operational, architectural, frontend contract, ML governance, testing, và documentation. Điểm chung là phần lớn debt đến từ control plane chứ không phải từ model core.

Debt class	Rank	Bằng chứng
Security debt	Critical	không có auth/authz, CORS rộng, Grafana password mặc định, evaluate public
Operational debt	High	batch status mock, evaluation state trong memory, không có alert routing, không có tracing
Architectural debt	High	một process sở hữu public API và ML orchestration nặng; không có worker isolation cho explain hay batch
Frontend contract debt	High	UI language selector vượt backend support; explicit lang không đi hết predict path
ML governance debt	Medium	artifact integrity và promotion checks mọi o mục ngam định
Testing debt	Medium	Python coverage tốt, nhưng frontend và security tests mong
Documentation debt	Medium	README và docs cụ thể implementation o endpoint và batch behavior

17 Section 17 - Scalability Analysis

17.1 Application Scalability

Application scalability bị giới hạn bởi một FastAPI process sở hữu cả API lại model orchestration. Khi concurrency tăng, contention sẽ xuất hiện trước tiên ở CPU, rồi đến event loop và memory pressure.

17.2 Model Scalability

Model scalability bị chi phối bởi sentiment inference cộng thêm ABSA và SHAP. Nếu workload nghiêng về explain hoặc batch lớn, chi phí per request tăng đáng kể ngay cả khi ONNX đã giảm thời gian forward pass.

17.3 Operational Scalability

Operational scalability bị giới hạn bởi thiếu auth, thiếu alert routing, và batch/evaluation state không bền. Tức là mở rộng team hay traffic đều làm tăng risk vận hành nếu control plane không đổi.

17.4 Primary Bottlenecks

Nút thắt đầu tiên là SHAP/explain. Nút thắt thứ hai là batch inference đồng bộ. Nút thắt dài hạn là single-process ownership của backend cho cả serving và heavy ML workflow.

17.5 Scalability View

- application scalability bị giới hạn bởi một in-process model owner và không có external queue
- model scalability bị giới hạn bởi chi phí ABSA và SHAP, không chỉ sentiment inference
- infrastructure scalability bị giới hạn trong khuôn khổ Compose hơn là managed orchestration

- operational scalability bị giới hạn bởi access control yêu cầu và workflow state không bền
- data scalability tạm đủ cho kiểu CSV và DVC hiện tại, nhưng governance vẫn thủ công
- team scalability bị ảnh hưởng bởi kiến thức tập trung vào `src/model/`, `src/scripts/`, và deployment config

18 Section 18 - Architectural Assessment

18.1 Scorecard

Scorecard tổng hợp cho thấy ML lifecycle mạnh hơn security và operations. Đó là dấu hiệu của một hệ thống research-to-pilot đã trưởng thành, nhưng chưa đủ production hardening.

Dimension	Score	Justification
Architecture quality	7/10	layering rõ, artifact flow hợp lý, nhưng runtime vẫn tập trung quá nhiều
Maintainability	6/10	backend tách module khá ổn; frontend components lộn và mixed concerns
Reliability	5/10	có health và tests, nhưng durability boundaries yếu
Security	3/10	thiếu nhiều control chặn sản xuất
Performance	6/10	ONNX và batching giúp nhiều; explain và ABSA vẫn dật
Observability	6/10	có metrics và dashboards; tracing và alert routing chưa có
Extensibility	7/10	model mode abstraction và task configs demo rõ ràng
Operational readiness	5/10	local integrated stack tốt; enterprise controls chưa đủ
Testing maturity	7/10	Python coverage rõ ràng; frontend và security tests hạn chế
ML lifecycle maturity	8/10	DVC, MLflow, training, evaluation, export, benchmark đều có

19 Section 19 - Recommendations

19.1 Immediate Improvements

Ưu tiên đầu tiên là khóa auth/authz, giảm surface công khai của evaluate, và chuẩn hóa batch status semantics. Những thay đổi này có tác động cao nhất đến risk giảm ngay.

19.2 Short- to Long-term Roadmap

Roadmap ngắn hạn nên tập trung vào rate limiting, secret hardening, và logging chuẩn hơn. Roadmap trung hạn nên tách batch/explain sang worker, còn dài hạn thì cần deployment orchestration mạnh hơn Compose.

Horizon	Vấn đề	Bằng chứng	Priority	Complexity	Expected outcome
Immediate	public API và admin routes không auth	không có auth trong backend hay gateway	Critical	Medium	có trust boundary tối thiểu và pilot an toàn hơn
Immediate	batch status contract gay hieu nam	mocked <code>/batch_status</code> endpoint	High	Low	loại bỏ async semantics giả hoặc triển khai jobs that tang do dung và user trust
Immediate	frontend/backend language drift	UI hiện nhiều language, predict path bỏ explicit lang	High	Medium	
Short-term	tách heavy evaluation path	<code>/evaluate</code> chạy trong public app process	High	Medium	giảm abuse và ổn định runtime

Horizon	Van de	Bằng chứng	Priority	Complexity	Expected outcome
Short-term	harden secret handling	build args va Grafana password mac dinh	High	Medium	giam credential exposure
Short-term	them rate limiting va file controls	không có throttling; up-load validation tối thiểu	High	Medium	resilience tot hon khi bi mis-use
Medium-term	dua batch va explain nang sang background workers	hiện tại là synchronous in-process CPU work	Medium	High	latency de doan duoc va scale tot hon
Medium-term	them artifact integrity checks va promotion workflow	không có checksum hay signed promotion path	Medium	Medium	model release an toan hon
Long-term	vuot qua Compose-only deployment	không có managed roll-out strategy hay secret platform	Medium	High	operational scalability va deployment safety tot hon

20 Section 20 - Final Technical Conclusion

20.1 Major Strengths

- full repository-level ML lifecycle được implement thật
- runtime và training concerns được tách đủ rõ cho đội nhỏ
- contracts, tests, monitoring, và export paths giúp system audit được
- ONNX serving path, batch optimization, và MLflow integration la nhung quyet dinh thuc dung

20.2 Major Weaknesses

- không có authentication hay authorization
- batch async semantics bị mock, evaluation state không bền
- frontend/backend contract drift quanh language handling và batch behavior
- heavy inference features chia sẻ cùng một public process, không có workload isolation

20.3 Critical Risks

- su dung trai phep cac endpoints dat tien hoac co tinh chat quan tri
- vận hành hiệu sai do batch status không phản ánh thực tế
- performance sup do khi traffic nghieng ve explain hoac batch
- artifact va secret handling yeu trong build va deployment

20.4 Production Readiness and Final Verdict

Chỉ dựa trên bằng chứng implementation, hệ thống **chưa sẵn sàng cho production enterprise**. Tuy vậy, đây là một Advanced MVP đáng tin và là một nền tảng demo hoặc pilot kiểm soát tốt. Kiến trúc chạy được end-to-end, ML lifecycle có thật, và repository có giá trị kỹ thuật rõ ràng. Khoảng cách

hiện tại không nằm ở chỗ thiếu feature; nó nằm ở chỗ thiếu control plane cần thiết để vận hành các feature đó một cách an toàn, bền vững, và mở rộng được.